

② 第二周 —— 单片机有关中断内容

☺ 关于led的中断

1. 实验现象为按下s4, l2亮; 按下s5, l1亮;
再次按下s4, l2灭; 再次按下s5, l1灭

```
1  #include <STC15F2K60S2.H>
2  #include "LED.h"
3  #include "Init.h"
4  #include <z_init.h>
5  //led专用变量
6  unsigned char ucled;
7
8
9  void main()
10 {
11
12     z_init();
13     per_init();//外设初始化
14     while (1)
15     {
16
17
18     }
19
20 }
21
22
23 void isr1(void) interrupt 0
24 {
25
26     //s5按下之后led1进行反转
27     ucled^=0X01;//相同为0, 不同为1
28     led_disp(ucled);
29
30 }
31
32
33 void isr2(void) interrupt 2
34 {
35
36     //s4按下之后led2进行反转
37     ucled^=0X02;//相同为0, 不同为1
38     led_disp(ucled);
39 }
```

细讲

☐ 外部中断控制 LED (STC15F2K60S2)

1. 一、实验功能说明

- 按键 S5 按下 → LED1 翻转
 - 按键 S4 按下 → LED2 翻转
 - 程序采用 **外部中断方式**，主循环为空
-

2. 二、程序整体结构

2.1 1 主函数 `main`

```
1 void main()
2 {
3     z_init();    // 中断初始化
4     per_init();  // 外设初始化
5     while (1)
6     {
7     }
8 }
```

Fence 2

说明：

- `main()` 中只负责 **初始化**
 - 不写按键扫描、不写 LED 控制逻辑
 - 所有响应行为交由 **中断服务函数** 完成
👉 属于 **中断驱动程序结构**
-

3. 三、LED 状态变量设计

```
1 unsigned char ucled;
```

Fence 3

3.1 1 含义说明

- `ucled` 是一个 **8 位无符号变量**
- 用来统一保存 LED 的状态
- 每一位对应一个 LED

3.2 2 位与 LED 的对应关系

位	掩码	LED
bit0	0x01	LED1
bit1	0x02	LED2
bit2	0x04	LED3
bit3	0x08	LED4
bit4	0x10	LED5
bit5	0x20	LED6
bit6	0x40	LED7
bit7	0x80	LED8

Table 1

👉 ucled 本质上是一个 LED 状态寄存器

4. 四、外部中断服务函数分析

4.1 4.1 外部中断 0（INT0）

```
1 void isr1(void) interrupt 0
2 {
3     ucled ^= 0x01;    // 翻转 LED1 (bit0)
4     led_disp(ucled); // 更新 LED 显示
5 }
```

Fence 4

说明：

- interrupt 0 固定表示 外部中断 0（INT0）
- 在蓝桥杯开发板上，INT0 接 按键 S5
- ^= 为异或运算，可实现 位翻转

4.2 4.2 外部中断 1（INT1）

```
1 void isr2(void) interrupt 2
2 {
3     ucled ^= 0x02;    // 翻转 LED2 (bit1)
4     led_disp(ucled); // 更新 LED 显示
5 }
```

Fence 5

说明：

- interrupt 2 固定表示 外部中断 1（INT1）
- 在蓝桥杯开发板上，INT1 接 按键 S4

5. 五、外部中断初始化函数 `z_init`

```
1 void z_init(void)
2 {
3     IT0 = 1;    // INT0 下降沿触发
4     EX0 = 1;    // 允许外部中断0
5
6     IT1 = 1;    // INT1 下降沿触发
7     EX1 = 1;    // 允许外部中断1
8
9     EA = 1;     // 打开总中断
10 }
```

Fence 6

5.1 5.1 各语句作用（人话版）

○ `IT0 = 1`

→ INT0 使用 **下降沿触发**（按键按下瞬间触发）

○ `EX0 = 1`

→ 允许 INT0 中断进入

○ `IT1 = 1`

→ INT1 使用下降沿触发

○ `EX1 = 1`

→ 允许 INT1 中断进入

○ `EA = 1`

→ 打开 **总中断开关**（所有中断的总电源）

✦ **EA 不开，所有中断都无效**

6. 六、 `.h` 文件的作用

```
1 #include <STC15F2K60S2.H>
2 void z_init(void);
```

Fence 7

说明：

○ `.h` 文件只负责 **函数声明**

○ 让其他 `.c` 文件（如 `main.c`）能调用 `z_init()`

○ 不包含具体功能实现

7. 七、关键知识点总结（蓝桥杯必会）

1. `interrupt 0` → 外部中断 0（INT0，P3.2）

2. `interrupt 2` → 外部中断 1（INT1，P3.3）

3. `ucled` 常用于 **8 个 LED 的统一管理**

4. ^= 是 LED 翻转的常用位运算

5. 外部中断初始化三步：

- 设触发方式 (ITx)
- 开中断允许 (EXx)
- 开总中断 (EA)

8. 八、一句话总结（复习用）

本程序利用外部中断实现按键控制 LED，通过一个 8 位变量统一管理 LED 状态，中断初始化只需设置触发方式、允许位和总中断即可。

🕒 流水灯和呼吸灯

```
1  #include <STC15F2K60S2.H>
2  #include "LED.h"
3  #include "Init.h"
4  #include "timer.h"
5  unsigned long int ms_Tick; //滴答定时器
6  unsigned char ucled; //led专用变量
7  unsigned char count;
8  unsigned char count_com;
9  unsigned char sign=1;
10
11
12
13 void main()
14 {
15
16
17     per_init();
18     Timer1_Init();
19
20
21
22
23
24
25     while(1)
26     {
27         //if(ms_Tick==500)//每500ms进入一次, 0.5s进入一次
28         //{
29             //流水灯
30             if(count<8)
31             {
32                 ucled=0x01<<count; //每次左移一位
33                 count++;
34             }
35             //
36             //
37             else
38             {
39                 count=0;
40                 ms_Tick=0;
41             }
42             //
43         }
44
45         //呼吸灯
46         if(ms_Tick==500)//每500ms进入一次, 0.5s进入一次
47         {
48             if(sign==1)
49             {
50                 count_com++;
51             }
52
53             else if(sign==0)
54             {
55                 count_com--;
56             }
57
58             if(count_com==10)
59             {
60                 sign=0;
61             }
62             else if(count_com==0)
63             {
64                 sign=1;
65             }
66         }
67     }
```



```

66         }
67
68         ms_Tick=0;
69
70     }
71 }
72
73 }
74 void tim_isr() interrupt 3
75 {
76     //每1ms进入一次
77     ms_Tick++;
78     count++;
79     if(count==10) count=0;
80     if(count<count_com)
81         ucled=0xff;
82     else
83         ucled=0x00;
84
85     led_disp(ucled);
86
87 }
88

```

🗨 细节

☺ LED 灯效程序解析（流水灯 + 呼吸灯）

1. 一、整体运行框架

核心逻辑：

定时器 1 每 1ms 触发一次中断 → main 中根据 `ms_Tick` 判断节拍 → 决定 LED 的显示状态

- **Timer1 中断**：计时 + 控制 LED 明暗
- **main 循环**：根据计时变量 `ms_Tick`，决定当前 LED 应该怎么显示

这种结构属于 中断驱动 + 状态机控制 LED，主循环无需阻塞。

2. 二、流水灯部分

2.1 1 流水灯核心代码

```
1 | if(ms_Tick == 500) // 每 500ms 执行一次
2 | {
3 |     if(count < 8)
4 |     {
5 |         ucled = 0x01 << count; // 每次左移一位, 点亮下一个 LED
6 |         count++;
7 |     }
8 |     else
9 |     {
10 |         count = 0; // 回到第一个 LED
11 |     }
12 |     ms_Tick = 0; // 重置计时
13 | }
```

Fence 9

2.2 2 代码原理解析

1. 定时控制

```
1 | if(ms_Tick == 500)
```

Fence 10

- Timer1 每 1ms 中断一次
- `ms_Tick` 达到 500 → 执行一次流水灯逻辑
- 500ms = 0.5秒 → LED 每 0.5 秒移动一次

1. 移位控制 LED

```
1 | | ucled = 0x01 << count;
```

Fence 11

- 把二进制的 1 左移 count 位
- 对应点亮第 count+1 个 LED

count	ucled (二进制)	点亮 LED
0	0000 0001	LED1
1	0000 0010	LED2
2	0000 0100	LED3
...	...	...

Table 2

1. 循环回到起点

```
1 | | if(count >= 8) count = 0;
```

Fence 12

- LED1 → LED8 → 再回到 LED1
- 实现“循环流水灯”

2.3 3 总结一句话

流水灯 = 定时器节拍 + 左移操作 → LED 依次点亮，循环显示

3. 三、呼吸灯部分

3.1 1 main 中控制亮度变化

```
1 | | if(ms_Tick == 50) // 每 50ms 执行一次
2 | | {
3 | |     if(sign == 1) // 增亮
4 | |         count_com++;
5 | |     else // 变暗
6 | |         count_com--;
7 | |
8 | |     if(count_com == 10) // 达到最大亮度，开始变暗
9 | |         sign = 0;
10 | |     else if(count_com == 0) // 达到最暗，开始增亮
11 | |         sign = 1;
12 | |
13 | |     ms_Tick = 0; // 重置计时
14 | | }
```

Fence 13

3.2 2 核心原理（简单理解版）

1.呼吸灯并不是 LED 真正亮度变化

- LED 只有 亮/灭
- “半亮” 是通过快速闪烁，让人眼看起来亮度变化（软件 PWM）
- PWM 是啥？（别被名字吓到）

先给你一句“考试级解释”

PWM 是通过快速开和关，改变“亮的时间比例”，来控制亮度的一种方法。

2.count_com

- 决定当前周期内 LED “亮” 的次数
- 越大 → 看起来越亮
- 越小 → 看起来越暗

3.sign

- 控制亮度方向
- 1 → 增亮
- 0 → 变暗

3.3 3 Timer1 中断真正控制 LED

```
1 void tim_isr() interrupt 3
2 {
3     ms_Tick++;          // 时间 +1ms
4     count++;            // PWM 周期计数
5     if(count == 10) count = 0;
6
7     if(count < count_com)
8         ucled = 0xff;    // LED 亮
9     else
10        ucled = 0x00;    // LED 灭
11
12    led_disp(ucled);     // 输出到 LED
13 }
```

Fence 14

3.3.1 核心解释

变量	含义
count	当前 PWM 小周期计数（0~9）
count_com	当前占空比（亮多少次）
ucled	LED 状态寄存器
led_disp()	真正控制 LED 输出

Table 3

原理：

- 在一个 10ms 的小周期里，前 count_com 次亮 → 后面灭
- 每 50ms 更新 count_com → 实现呼吸效果

3.4 4 呼吸灯一句话总结

通过定时器中断控制 LED 在每个小周期内亮灭比例，再周期性改变亮的次数，让 LED 看起来由暗到亮再由亮到暗。

4. 四、流水灯 vs 呼吸灯对比

特点	流水灯	呼吸灯
控制方式	移位操作	软件 PWM（亮灭比例控制）
时间基准	ms_Tick 500ms	ms_Tick 50ms
核心变量	count	count_com + sign
LED 显示规律	依次点亮	亮度渐变（模拟呼吸）
可视效果	灯光移动	灯光呼吸

Table 4

5. 五、学习重点（新手 / 蓝桥杯）

1.流水灯

- 会用移位控制 LED
- 会用定时器做节拍

2.呼吸灯

- 理解“快闪 → 人眼看到亮度变化”
- 会用 `count_com` 改亮灭次数
- 能改最大亮度和速度

3.不需要掌握

- PWM 公式、电气原理
- 复杂波形和占空比计算

能写、能改、能解释一句话 = 完全够用